

LAPORAN PRAKTIKUM – HEAP TREE

Laporan ini disusun untuk memenuhi Tugas Ke Sebelas Praktikum Algoritma &

Pemrograman

Dosen Pengampu : Dr. Ionia Veritawati, S.Si., MT.



Oleh :

Nama : Khairul Adam Efendi

NPM : 4525210035

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS PANCASILA

2025/2026

Soal Heap Tree_01 (nama file: PASD11-01.cpp):

- **Source Code**

1. Berdasarkan contoh no.1 HEAP TREE, buat pengurutan secara descending
(nama file : PASD11-01.cpp):

```
#include<iostream>

using namespace std;

void HEAP(int canArray[],int n,int i){
    int temp,canBesar=i,kiri=2*i+1,kanan=2*i+2;
    if(kiri<n&&canArray[kiri]<canArray[canBesar])
        canBesar=kiri;
    if(kanan<n&&canArray[kanan]<canArray[canBesar])
        canBesar=kanan;
    if(canBesar!=i){
        temp=canArray[i];
        canArray[i]=canArray[canBesar];
        canArray[canBesar]=temp;
        HEAP(canArray,n,canBesar);
    }
}

void sortHEAP(int canArray[],int n){
    int temp;
    for(int i=n/2-1;i>=0;i--){
        HEAP(canArray,n,i);
    }
    for(int i=n-1;i>=0;i--){
        temp=canArray[0];
        canArray[0]=canArray[i];
        canArray[i]=temp;
        HEAP(canArray,i,0);
    }
}
```

```
int main(){
    int canArray[]={22,7,66,28,11,63,24,12,77,99};
    int n=10,i;
    cout<<"Menampilkan data sebelum diurutkan"<<endl;
    cout<<"~~~~~"<<endl;
    for(i=0;i<n;i++)
        cout<<canArray[i]<<" ";
    cout<<endl;
    sortHEAP(canArray,n);
    cout<<endl;
    cout<<"Menampilkan data setelah diurutkan"<<endl;
    cout<<"~~~~~"<<endl;
    for(i=0;i<n;i++)
        cout<<canArray[i]<<" ";
    cin.get();
}
```

- SS Program

```
#include<iostream>
using namespace std;

void HEAP(int canArray[],int n,int i){
    int temp,canBesar=i,kiri=2*i+1,kanan=2*i+2;
    if(kiri<n&&canArray[kiri]<canArray[canBesar])
        canBesar=kiri;
    if(kanan<n&&canArray[kanan]<canArray[canBesar])
        canBesar=kanan;
    if(canBesar!=i){
        temp=canArray[i];
        canArray[i]=canArray[canBesar];
        canArray[canBesar]=temp;
        HEAP(canArray,n,canBesar);
    }
}

void sortHEAP(int canArray[],int n){
    int temp;
    for(int i=n/2-1;i>=0;i--){
        HEAP(canArray,n,i);
    }
    for(int i=n-1;i>=0;i--){
        temp=canArray[0];
        canArray[0]=canArray[i];
        canArray[i]=temp;
        HEAP(canArray,i,0);
    }
}

int main(){
    int canArray[]={22,7,66,28,11,63,24,12,77,99};
    int n=10,i;
    cout<<"Menampilkan data sebelum diurutkan"<<endl;
    cout<<"~~~~~"<<endl;
    for(i=0;i<n;i++){
        cout<<canArray[i]<<" ";
    }
    cout<<endl;
    sortHEAP(canArray,n);
    cout<<endl;
    cout<<"Menampilkan data setelah diurutkan"<<endl;
    cout<<"~~~~~"<<endl;
    for(i=0;i<n;i++){
        cout<<canArray[i]<<" ";
    }
    cin.get();
}
```

- SS Running Program

```
D:\ASD SMT2\ASD-A-25-26\PRAK11_HeapTree_Asprak01_4525210035_Khairul Adam Efendi>PAS011-01
Menampilkan data sebelum diurutkan
~~~~~
22 7 66 28 11 63 24 12 77 99

Menampilkan data setelah diurutkan
~~~~~
99 77 66 63 28 24 22 12 11 7
```

Soal Heap Tree_06 (nama file: PASD11-06.cpp):

- **Source Code**

6. Perbaiki kesalahan coding dibawah ini (nama file : PASD11-06.cpp):

```
#include<iostream>
#include<cstdlib>
using namespace std;

struct Node{
    struct Node *Left;
    char INFO;
    struct Node *Right;
};

typedef struct Node Simpul;

Simpul *Root,*P,*Current;
Simpul *Q[129];

void Inisialisasi(){
    Root=NULL;
    P=NULL;
    Current=NULL;
    for(int i=0;i<129;i++){
        Q[i]=NULL;
    }
}

void BuatSimpul(char X){
    P=new Simpul;
    if(P!=NULL){
        P->INFO=X;
        P->Left=NULL;
        P->Right=NULL;
    }
}
```

```

    }
    else{
        cout<<"Memory Heap Full"<<endl;
        exit(1);
    }
}

void BuatSimpulAkar(){
    if(Root==NULL){
        if(P!=NULL){
            Root=P;
            Root->Left=NULL;
            Root->Right=NULL;
            cout<<"Root Berhasil Dibuat"<<endl;
        }
        else{
            cout<<"Simpul Belum Dibuat"<<endl;
        }
    }
    else{
        cout<<"Root Sudah Ada"<<endl;
    }
}

bool IsEmpty(){
    if(Root==NULL)
        return true;
    else
        return false;
}

Simpul *Alokasi(char X){
    Simpul *NodeBaru;

```

```

NodeBaru=new Simpul;
if(NodeBaru!=NULL){
    NodeBaru->INFO=X;
    NodeBaru->Left=NULL;
    NodeBaru->Right=NULL;
}
return NodeBaru;
}

void InsertUrutNomor(){
    int depan=1;
    int belakang=1;
    char X;

    Q[belakang]=Root;

    while(depan<=belakang){
        Current=Q[depan];

        cout<<"Masukkan Anak Kiri dari "<<Current->INFO<<" (0 jika
tidak ada) : ";
        cin>>X;
        if(X!='0'){
            Current->Left=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Left;
        }

        cout<<"Masukkan Anak Kanan dari "<<Current->INFO<<" (0
jika tidak ada) : ";
        cin>>X;
        if(X!='0'){
            Current->Right=Alokasi(X);

```

```

        belakang++;
        Q[belakang]=Current->Right;
    }

    depan++;
}
}

void BacaUrutNomor(){
    int depan=1;
    int belakang=1;

    Q[belakang]=Root;

    cout<<endl;
    cout<<"===== "<<endl;
    cout<<"ISI BINARY TREE (LEVEL ORDER)"<<endl;
    cout<<"===== "<<endl;

    while(depan<=belakang){
        Current=Q[depan];

        if(Current!=NULL){
            cout<<Current->INFO<<" ";

            if(Current->Left!=NULL){
                belakang++;
                Q[belakang]=Current->Left;
            }

            if(Current->Right!=NULL){
                belakang++;
                Q[belakang]=Current->Right;
            }
        }
    }
}

```



```

        }

    }

    depan++;

}

cout<<endl;
}

void PreOrder(Simpul *Root){
    if(Root!=NULL){
        cout<<Root->INFO<<" ";
        PreOrder(Root->Left);
        PreOrder(Root->Right);
    }
}

void InOrder(Simpul *Root){
    if(Root!=NULL){
        InOrder(Root->Left);
        cout<<Root->INFO<<" ";
        InOrder(Root->Right);
    }
}

void PostOrder(Simpul *Root){
    if(Root!=NULL){
        PostOrder(Root->Left);
        PostOrder(Root->Right);
        cout<<Root->INFO<<" ";
    }
}

```

```

int HitungNode(Simpul *Root){
    if(Root==NULL)
        return 0;
    return 1+HitungNode(Root->Left)+HitungNode(Root->Right);
}

int HitungDaun(Simpul *Root){
    if(Root==NULL)
        return 0;
    if(Root->Left==NULL && Root->Right==NULL)
        return 1;
    return HitungDaun(Root->Left)+HitungDaun(Root->Right);
}

int TinggiTree(Simpul *Root){
    if(Root==NULL)
        return 0;

    int kiri=TinggiTree(Root->Left);
    int kanan=TinggiTree(Root->Right);

    if(kiri>kanan)
        return kiri+1;
    else
        return kanan+1;
}

int main(){
    char X;

    Inisialisasi();

    cout<<"===== "<<endl;

```

```

cout<<"    PROGRAM BINARY TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Masukkan Root : ";
cin>>X;

    BuatSimpul(X);
    BuatSimpulAkar();

cout<<endl;
InsertUrutNomor();

cout<<endl;
BacaUrutNomor();

cout<<endl;
cout<<"===== "<<endl;
cout<<"PREORDER"<<endl;
cout<<"===== "<<endl;
PreOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INORDER"<<endl;
cout<<"===== "<<endl;
InOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"POSTORDER"<<endl;

```

```
cout<<"===== "<<endl;
PostOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INFORMASI TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Jumlah Node : "<<HitungNode(Root)<<endl;
cout<<"Jumlah Daun : "<<HitungDaun(Root)<<endl;
cout<<"Tinggi Tree : "<<TinggiTree(Root)<<endl;

cout<<endl;
system("pause");
return 0;
}
```

- SS Program

```
#include<iostream>
#include<cstdlib>
using namespace std;

struct Node{
    struct Node *Left;
    char INFO;
    struct Node *Right;
};

typedef struct Node Simpul;

Simpul *Root,*P,*Current;
Simpul *Q[129];

void Inisialisasi(){
    Root=NULL;
    P=NULL;
    Current=NULL;
    for(int i=0;i<129;i++){
        Q[i]=NULL;
    }
}

void BuatSimpul(char X){
    P=new Simpul;
    if(P!=NULL){
        P->INFO=X;
        P->Left=NULL;
        P->Right=NULL;
    }
    else{
        cout<<"Memory Heap Full"<<endl;
        exit(1);
    }
}

void BuatSimpulAkar(){
    if(Root==NULL){
        if(P!=NULL){
            Root=P;
            Root->Left=NULL;
            Root->Right=NULL;
            cout<<"Root Berhasil Dibuat"<<endl;
        }
        else{
            cout<<"Simpul Belum Dibuat"<<endl;
        }
    }
    else{
```

```

        cout<<"Root Sudah Ada"<<endl;
    }
}

bool IsEmpty() {
    if (Root==NULL)
        return true;
    else
        return false;
}

Simpul *Alokasi(char X) {
    Simpul *NodeBaru;
    NodeBaru=new Simpul;
    if (NodeBaru!=NULL) {
        NodeBaru->INFO=X;
        NodeBaru->Left=NULL;
        NodeBaru->Right=NULL;
    }
    return NodeBaru;
}

void InsertUrutNomor() {
    int depan=1;
    int belakang=1;
    char X;

    Q[belakang]=Root;

    while (depan<=belakang) {
        Current=Q[depan];

        cout<<"Masukkan Anak Kiri dari "<<Current->INFO<<" (0 jika tidak ada) : ";
        cin>>X;
        if (X!='0') {
            Current->Left=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Left;
        }

        cout<<"Masukkan Anak Kanan dari "<<Current->INFO<<" (0 jika tidak ada) : ";
        cin>>X;
        if (X!='0') {
            Current->Right=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Right;
        }

        depan++;
    }
}

```

```

}

void BacaUrutNomor() {
    int depan=1;
    int belakang=1;

    Q[belakang]=Root;

    cout<<endl;
    cout<<"===== "<<endl;
    cout<<"ISI BINARY TREE (LEVEL ORDER) "<<endl;
    cout<<"===== "<<endl;

    while (depan<=belakang) {
        Current=Q[depan];

        if (Current!=NULL) {
            cout<<Current->INFO<<" ";

            if (Current->Left!=NULL) {
                belakang++;
                Q[belakang]=Current->Left;
            }

            if (Current->Right!=NULL) {
                belakang++;
                Q[belakang]=Current->Right;
            }
        }

        depan++;
    }

    cout<<endl;
}

void PreOrder(Simpul *Root) {
    if (Root!=NULL) {
        cout<<Root->INFO<<" ";
        PreOrder (Root->Left);
        PreOrder (Root->Right);
    }
}

void InOrder(Simpul *Root) {
    if (Root!=NULL) {
        InOrder (Root->Left);
        cout<<Root->INFO<<" ";
        InOrder (Root->Right);
    }
}

```

```

}

void PostOrder(Simpul *Root){
    if(Root!=NULL){
        PostOrder(Root->Left);
        PostOrder(Root->Right);
        cout<<Root->INFO<<" ";
    }
}

int HitungNode(Simpul *Root){
    if(Root==NULL)
        return 0;
    return 1+HitungNode(Root->Left)+HitungNode(Root->Right);
}

int HitungDaun(Simpul *Root){
    if(Root==NULL)
        return 0;
    if(Root->Left==NULL && Root->Right==NULL)
        return 1;
    return HitungDaun(Root->Left)+HitungDaun(Root->Right);
}

int TinggiTree(Simpul *Root){
    if(Root==NULL)
        return 0;

    int kiri=TinggiTree(Root->Left);
    int kanan=TinggiTree(Root->Right);

    if(kiri>kanan)
        return kiri+1;
    else
        return kanan+1;
}

int main(){
    char X;

    Inisialisasi();

    cout<<"===== "<<endl;
    cout<<"          PROGRAM BINARY TREE"<<endl;
    cout<<"===== "<<endl;
    cout<<"Masukkan Root : ";
    cin>>X;

    BuatSimpul(X);
    BuatSimpulAkar();
}

```



```

cout<<endl;
InsertUrutNomor ();

cout<<endl;
BacaUrutNomor ();

cout<<endl;
cout<<"===== "<<endl;
cout<<"PREORDER"<<endl;
cout<<"===== "<<endl;
PreOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INORDER"<<endl;
cout<<"===== "<<endl;
InOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"POSTORDER"<<endl;
cout<<"===== "<<endl;
PostOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INFORMASI TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Jumlah Node   : "<<HitungNode (Root)<<endl;
cout<<"Jumlah Daun   : "<<HitungDaun (Root)<<endl;
cout<<"Tinggi Tree   : "<<TinggiTree (Root)<<endl;

cout<<endl;
system("pause");
return 0;

```

1 }

- **SS Running Program**

```
D:\ASD SMT2\ASD-A-25-26\PRAK11_HeapTree_Asprak01_4525210035_Khairul Adam Efendi>PASD11-06
=====
PROGRAM BINARY TREE
=====
Masukkan Root : 6
Root Berhasil Dibuat

Masukkan Anak Kiri dari 6 (0 jika tidak ada) : 8
Masukkan Anak Kanan dari 6 (0 jika tidak ada) : 5
Masukkan Anak Kiri dari 8 (0 jika tidak ada) : 2
Masukkan Anak Kanan dari 8 (0 jika tidak ada) : 3
Masukkan Anak Kiri dari 5 (0 jika tidak ada) : 4
Masukkan Anak Kanan dari 5 (0 jika tidak ada) : 5
Masukkan Anak Kiri dari 2 (0 jika tidak ada) : 4
Masukkan Anak Kanan dari 2 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 3 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 3 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 4 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 4 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 5 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 5 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 4 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 4 (0 jika tidak ada) : 0

=====
ISI BINARY TREE (LEVEL ORDER)
=====
6 8 5 2 3 4 5 4

=====
PREORDER
=====
6 8 2 4 3 5 4 5

=====
INORDER
=====
4 2 8 3 6 4 5 5

=====
POSTORDER
=====
4 2 3 8 4 5 5 6

=====
INFORMASI TREE
=====
Jumlah Node : 8
Jumlah Daun : 4
Tinggi Tree : 4

Press any key to continue . . .
```